

LÓGICA Y MATEMÁTICA DISCRETA

Inmaculada Gema Garcés Andrés

GRADO EN CIBERSEGURIDAD

Guía de estudio

viu Universidad Internacional de Valencia

Leyendas

Lectura obligatoria

Bibliografía recomendada

Enlace de interés

Ejemplo

abc Los términos resaltados a lo largo del contenido en color naranja se recogen en el apartado GLOSARIO.

Índice

Tema 1. Lógica proposicional 3

1.1. Proposiciones lógicas. 3

1.2. Conectores lógicos 5

1.2.1. Conjunción (AND, ∧) 5

1.2.2. Disyunción (OR, ∨) 5

1.2.3. Negación (¬) 5

1.2.4. Implicación (→) 6

1.2.5. Bicondicional (↔) 6

1.3. Tablas de verdad 6

1.4. Leyes lógicas fundamentales 7

1.5. Álgebra de Boole 8

1.6. Puertas lógicas 8

1.7. Circuitos lógicos 9

Tema 2. Lógica de predicados. 11

2.1. Predicados y universos 11

2.2. Cuantificadores 12

2.3. Operaciones y razonamiento en lógica de predicados 13

Tema 3. Recurrencia e inducción 16

3.1. Sucesiones y definiciones recursivas 16

3.2. Recurrencias lineales 17

3.3. Inducción 18

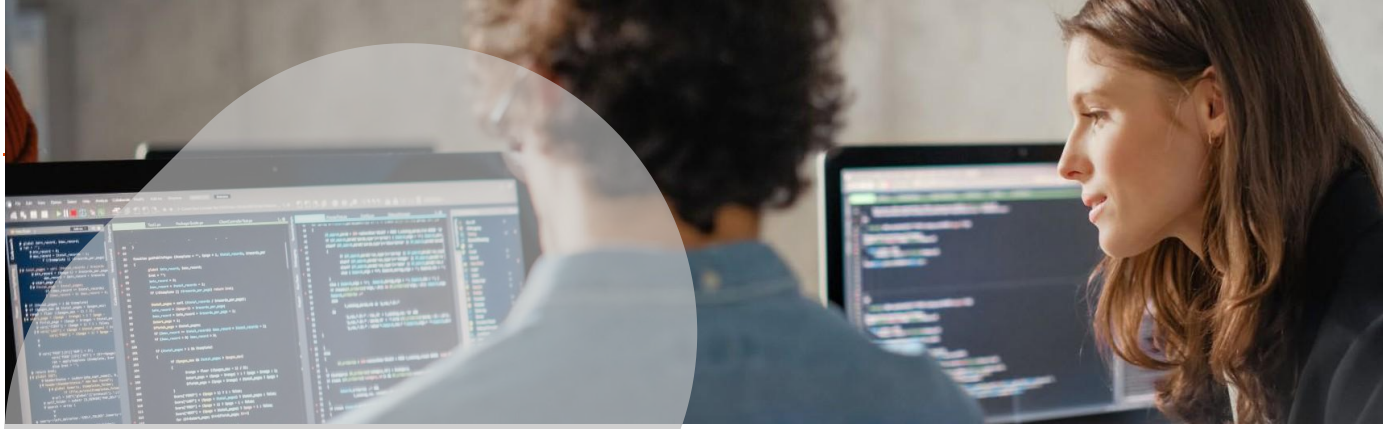
Tema 4. Teoría de grafos 20

4.1. Conceptos básicos de grafos. 20

4.2. Caminos, ciclos y propiedades de conectividad 22

4.3. Árboles, grafos ponderados y árboles generadores 23

4.4. Algoritmo de Dijkstra 23



Tema 1

Lógica proposicional

El contenido detallado de este tema es:

1.1. Proposiciones lógicas
1.2. Conectores lógicos
1.3. Tablas de verdad
1.4. Leyes lógicas fundamentales
1.5. Álgebra de Boole: definiciones y propiedades
1.6. Puertas lógicas (OR, AND, XOR, NOT, NOR, NAND, XNOR)
1.7. Circuitos lógicos y funciones booleanas

Vamos a ver qué se va a trabajar en cada uno de los apartados a grandes rasgos.

1.1. Proposiciones lógicas.

La lógica proposicional constituye una de las piedras angulares del pensamiento formal. Se trata del primer sistema lógico suficientemente estructurado para permitir la manipulación simbólica de enunciados y el análisis del razonamiento sin hacer referencia al contenido concreto de esos enunciados. Este nivel de abstracción convierte a la lógica proposicional en una herramienta esencial tanto en matemáticas, como en ingeniería informática, ciencia de datos, filosofía, lingüística y teoría de la computación.

El estudio de proposiciones surge de una necesidad fundamental: **distinguir argumentos válidos de inválidos**. Desde la Antigüedad, los filósofos buscaban un método sistemático para determinar si una conclusión se sigue necesariamente de unas premisas. Sin embargo, la lógica tradicional —basada en el lenguaje común— era ambigua y difícil de formalizar.

A finales del siglo XIX y principios del XX, Frege, Peano y Russell introdujeron una lógica basada en símbolos que permitía razonamientos rigurosos. Su primer nivel fue la lógica proposicional, donde:

- no se analizan sujetos ni predicados;
- solo interesa si las proposiciones son verdaderas o falsas;
- la estructura lógica se expresa mediante conectores.

Esta simplificación permite estudiar formas de razonamiento universales.

Una **proposición lógica** es un enunciado declarativo que puede ser evaluado como **verdadero (V)** o **falso (F)**.

Lo relevante es su **valor de verdad**, no la forma lingüística.



Ejemplos ilustrativos

- "Madrid es la capital de España." → V
- "2 es un número impar." → F
- "Si apruebo, entonces estudiar tuvo sentido." → depende de los valores asignados, pero sigue siendo una proposición.

No son proposiciones

- Imperativos: "Cierra la puerta."
- Preguntas: "¿Cómo estás?"
- Paradojas: "Esta frase es falsa.".

Nos encontramos con distintos tipos de proposiciones, los cuales se especifican a continuación.

- **Tautológicas:** Verdaderas en todo modelo posible.

  $$(p \vee \neg p)$$

  Ejemplo: "Todo múltiplo de 4 es múltiplo de 2."

- **Contradictorias:** Falsas en todos los modelos.

  $$(p \wedge \neg p)$$

  Ejemplo: "2 + 2 = 5."

- **Contingentes:** Verdaderas en algunos modelos y falsas en otros.

  $p \wedge q$, depende de los valores de verdad de p y q.

  Ejemplo: "Mañana hará sol."

Diferenciar entre tautologías, contradicciones y contingencias permite:

- conocer si un razonamiento es válido en todo caso,
- diseñar circuitos digitales que funcionen bajo cualquier entrada,
- simplificar fórmulas para optimizar software o hardware,
- modelar situaciones en IA mediante lógica formal.

1.2. Conectores lógicos

Los conectores permiten crear proposiciones compuestas a partir de proposiciones simples. Su interpretación es puramente semántica (tablas de verdad) y también sintáctica (reglas formales).

1.2.1. Conjunción (AND, ∧)

$p \wedge q$ es verdadera solo cuando ambos lo son.

p	q	p ∧ q
V	V	V
V	F	F
F	V	F
F	F	F

1.2.2. Disyunción (OR, ∨)

$p \vee q$ es verdadera si al menos uno es verdadero.

p	q	p ∨ q
V	V	V
V	F	V
F	V	V
F	F	F

La disyunción usada en lógica proposicional es **inclusiva**, a diferencia de la disyunción natural del lenguaje cotidiano.

1.2.3. Negación (¬)

Invierte el valor de verdad.

$$\neg p = \begin{cases} V & \text{si } p = F, \\ F & \text{si } p = V. \end{cases}$$

1.2.4. Implicación (→)

La interpretación estándar sigue la "condición material":

$p \to q$ solo es falsa cuando $p = V$ y $q = F$.

p	q	p → q
V	V	V
V	F	F
F	V	V
F	F	V

La equivalencia clave:

$$p \to q \equiv \neg p \vee q$$

1.2.5. Bicondicional (↔)

$p \leftrightarrow q$ es verdadera cuando p y q tienen el mismo valor.

p	q	p ↔ q
V	V	V
V	F	F
F	V	F
F	F	V

1.3. Tablas de verdad

Son herramientas esenciales para **evaluar proposiciones compuestas**, verificar equivalencias y determinar validez lógica.

$$(p \vee q) \rightarrow p$$

p	q	p ∨ q	(p ∨ q) → p
V	V	V	V
V	F	V	V
F	V	V	F
F	F	F	V

No es tautología → es contingente.

Las tablas de verdad existen porque la lógica proposicional es un sistema completamente semántico, donde todo puede evaluarse mediante combinaciones finitas.

Esto permite:

- automatizar razonamientos;
- construir verificadores de software;
- detectar contradicciones y tautologías;
- diseñar circuitos que implementen funciones específicas.

1.4. Leyes lógicas fundamentales

Estas leyes permiten transformar expresiones lógicas y demostrar equivalencias.

Leyes algebraicas

- **Conmutatividad:** $p \vee q = q \vee p,\ \ p \wedge q = q \wedge p$
- **Asociatividad:** $(p \vee q) \vee r = p \vee (q \vee r)$
- **Distributividad:** $p \wedge (q \vee r) = (p \wedge q) \vee (p \wedge r)$

Leyes de De Morgan

$$\neg(p \vee q) = \neg p \wedge \neg q$$
$$\neg(p \wedge q) = \neg p \vee \neg q$$

Doble negación

$$\neg(\neg p) = p$$

Absorción

$$p \vee (p \wedge q) = p, p \wedge (p \vee q) = p$$

Estas leyes se usan para:

- simplificar circuitos;
- optimizar expresiones booleanas;
- demostrar validez simbólica;
- diseñar expresiones en lenguajes de programación.



Ejemplo con explicación intuitiva

$$\neg(\boldsymbol{p} \wedge \boldsymbol{q}) = \neg\boldsymbol{p} \vee \neg\boldsymbol{q}$$

Interpretación en la vida real: "No es cierto que (esté lloviendo y haga frío)" equivale a "No llueve o no hace frío".

1.5. Álgebra de Boole

Un conjunto B con operaciones ∧ y ∨ es un *álgebra de Boole* si cumple:

- Asociatividad
- Conmutatividad
- Distributividad
- Existencia de neutros: $x \vee 0 = x, x \wedge 1 = x$
- Existencia de complementarios: $x \vee \neg x = 1, x \wedge \neg x = 0$



Ejemplos

1. El conjunto {0,1} con operaciones básicas de suma y producto booleano.
2. El conjunto de vectores binarios B^n.

Una función booleana de n variables es: $f: B^n \to B$



Ejemplos

- $f(x, y) = x \vee y$
- $f(x, y, z) = x \cdot z + y$

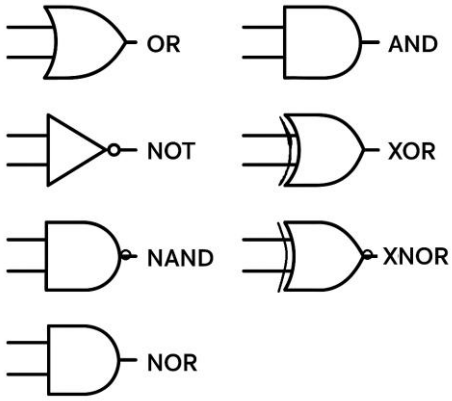
Estas funciones corresponden directamente a circuitos electrónicos.

1.6. Puertas lógicas

Las puertas lógicas implementan físicamente las funciones booleanas. Aquí se indican sus definiciones:

- **Puerta OR:** $\boldsymbol{f(x, y) = x \vee y}$
- **Puerta AND:** $\boldsymbol{f(x, y) = x \wedge y}$

- **Puerta NOT:** $f(x) = \neg x$
- **Puerta XOR:** $f(x, y) = x \oplus y$
- **Puerta NAND:** $f(x, y) = \neg(x \wedge y)$
- **Puerta NOR:** $f(x, y) = \neg(x \vee y)$
- **Puerta XNOR:** $f(x, y) = \neg(x \oplus y)$



1.7. Circuitos lógicos

Un circuito lógico combina puertas para implementar cualquier función booleana.

Ejemplo

Dado el circuito:

$$f(x, y, z) = (x \wedge y) \vee z$$

Interpretación:

- El bloque AND procesa x y y.
- El resultado se combina mediante OR con z.

Este tipo de análisis es fundamental en diseño digital (Mano & Ciletti, 2017).

Lecturas obligatorias

- Villalpando Becerra, J. F., & García Sandoval, A. (2014). *Matemáticas discretas: Aplicaciones y ejercicios*. Larousse - Grupo Editorial Patria. Enlace aquí
  - Capítulo 2
- Tinoco del Valle, Jesús, Hernández Sastoque, Eric, Escorcia Caballero, Edgardo. **(2023).** *Introducción a la lógica matemática* (1st ed.). Editorial Unimagdalena. Enlace aquí
  - Capítulos 2 y 3

Lecturas obligatorias

- Gutiérrez García, I. (2013). *Matemáticas para informática*. Universidad del Norte Editorial. Enlace aquí
  - Capítulo 1
- Forero Cuervo, A. (2009;2011;). *Matemática estructural* (1st ed.). Universidad de Los Andes, Colombia. Enlace aquí
  - Capítulos A

Enlaces de interés

Simulador de lógica booleana (circuitos) – Logic.ly. Permite construir circuitos con puertas lógicas y ver su comportamiento en tiempo real.
https://logic.ly/demo

Truth Table Generator (Standford/otros). Genera tablas de verdad automáticamente para cualquier expresión.
https://web.stanford.edu/class/cs103/tools/truth-table-tool/

Bibliografía recomendada

Enderton, H. B. (2001). *A Mathematical Introduction to Logic*. Academic Press.

Huth, M. & Ryan, M. (2004). *Logic in Computer Science*. Cambridge University Press.

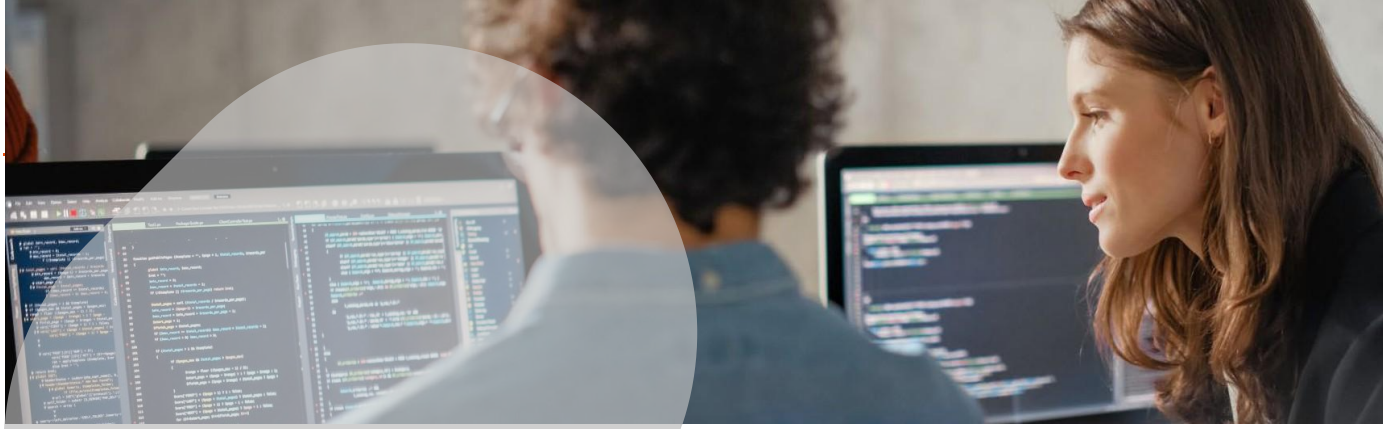
Ben-Ari, M. (2012). *Mathematical Logic for Computer Science*. Springer.

Rosen, K. H. (2012). *Discrete Mathematics and Its Applications*. McGraw-Hill.

Mendelson, E. (2015). *Introduction to Mathematical Logic*. Chapman & Hall.

Boolos, G., Burgess, J., & Jeffrey, R. (2007). *Computability and Logic*. Cambridge University Press.

Mano, M. & Ciletti, M. (2017). *Digital Design*. Pearson.



Tema 2

Lógica de predicados.

En este tema se van a abordar los siguientes puntos:

2.1. Predicados y universos.

2.2. Cuantificadores.

2.3. Operaciones y razonamiento en lógica de predicados.

2.1. Predicados y universos

La lógica proposicional permite razonar sobre enunciados completos, pero no es capaz de expresar adecuadamente relaciones o propiedades entre objetos.

Para ello necesitamos la **lógica de predicados** (o lógica de primer orden), una extensión más potente que:

- Usa **predicados** que expresan propiedades o relaciones.
- Introduce **cuantificadores** para referirse a “todos” o “algunos” objetos.
- Permite razonar con variables, términos y funciones.

Es la base formal de las matemáticas modernas, la inteligencia artificial simbólica, los lenguajes de programación lógicos y los sistemas de demostración automática.

Un **predicado** es una expresión lógica de la forma:

$$p(x_1, x_2, \dots, x_n)$$

que expresa una propiedad o relación entre los elementos x_i.

Cuando sustituimos las variables por valores concretos, obtenemos una proposición con valor de verdad.



Ejemplos

- $p(x)$: "x es un número par"
  - $p(4)$ es **verdadero**
  - $p(5)$ es **falso**
- $q(x, y)$: "x + y = 3"
  - $q(1,2)$ es **verdadero**
  - $q(1,1)$ es **falso**.

El universo U es el conjunto de valores que pueden tomar las variables de un predicado.



Ejemplos

- $Para\ p(x)$: "$x\ es\ mayor\ que\ 0$":
  - $U = R$
- $Para\ m(x)$: "$x\ es\ mamífero$":
  - $U\ podría\ ser$ "$todos\ los\ seres\ vivos$" o "$todos\ los\ animales$"

2.2. Cuantificadores

Los cuantificadores permiten construir frases sobre un conjunto completo de objetos (todos) o sobre la existencia de alguno.

Cuantificador universal

Se interpreta como **"para todo"**.



Ejemplo verbal:

- "Todo número entero es menor que su cuadrado para números mayores que 1."
- "Todos los estudiantes aprobados están en la lista A."

Una afirmación universal exige que **no haya excepciones**.

Cuantificador existencial

Se interpreta como **"existe al menos uno"**.

Ejemplos:

- "Existe un número que es igual a sí mismo menos 1."
- "Existe un alumno que no entregó la práctica."

Una afirmación existencial solo necesita **un caso verdadero**.

El orden importa

Decir "para cada x existe un y" no es lo mismo que "existe un y que funciona para todos los x".

Ejemplo sencillo:

- "Para todo número natural existe otro que es mayor" → Verdadero
- "Existe un número natural mayor que todos los demás" → Falso

Negación de cuantificadores

Negar un cuantificador cambia su significado profundamente:

- Negar un "para todo" significa: "hay al menos una excepción".
- Negar un "existe" significa: "ningún objeto cumple la propiedad".

Ejemplos verbales:

- "No todos los alumnos estudiaron" equivale a "Algún alumno no estudió".
- "No existe un número negativo mayor que 0" equivale a "Todos los números negativos son menores o iguales que 0".

2.3. Operaciones y razonamiento en lógica de predicados

Sustitución

Consiste en reemplazar variables por valores o expresiones.

Ejemplo:

Si el predicado es "x es mayor que y", y sustituimos x por 5, queda:

- "5 es mayor que y".

La sustitución permite transformar expresiones en otras más concretas y es una operación básica en razonamiento formal.

Unificación

La unificación es el proceso de hacer que dos expresiones que parecen distintas puedan ser consideradas iguales mediante sustituciones adecuadas.

Ejemplo en palabras:

- "x es amigo de Ana" y "Juan es amigo de Ana" pueden hacerse coincidir si sustituimos x por Juan.

Ejemplo de no unificación:

- "x es padre de x"
- "x es padre de y"

Estas no pueden hacerse coincidir si asumimos que un padre no puede ser la misma persona que su hijo.

Cómo se combinan predicados y cuantificadores

La lógica de predicados permite construir frases complejas:

- "Todo número tiene un sucesor."
- "Existe un estudiante que ha aprobado todas las asignaturas."
- "Si una persona es madre de otra, entonces existe una relación familiar entre ambas."

Estas expresiones pueden combinarse usando conectores como "y", "o", "si… entonces…", igual que en la lógica proposicional.

Ejemplo completo integrador:

Enunciado:

"Todo número natural tiene un sucesor, pero no existe ningún número natural que sea mayor que todos."

Formalización:

$$\forall x \in \mathbb{N},\ \exists y \in \mathbb{N}: y = x + 1$$

$$\neg\exists y \in \mathbb{N},\ \forall x \in \mathbb{N}: y > x$$

Interpretación:

- Hay infinitos naturales (primer enunciado).
- No hay un "máximo natural".

Lecturas obligatorias

- Tinoco del Valle, Jesús, Hernández Sastoque, Eric, Escorcia Caballero, Edgardo. (2023). *Introducción a la lógica matemática* (1st ed.). Editorial Unimagdalena. Enlace aquí
  - Capítulo 6

Lecturas obligatorias

- Mendelson, E. (2015). *Introduction to Mathematical Logic*. Enlace aquí
  - Capítulo 2
- Enderton, H. (2001). *A Mathematical Introduction to Logic*. Enlace aquí
  - Capítulo 1

Enlaces de interés

Simulador de predicados (Stanford):
https://web.stanford.edu/class/cs103/tools/truth-table-tool/

Bibliografía recomendada

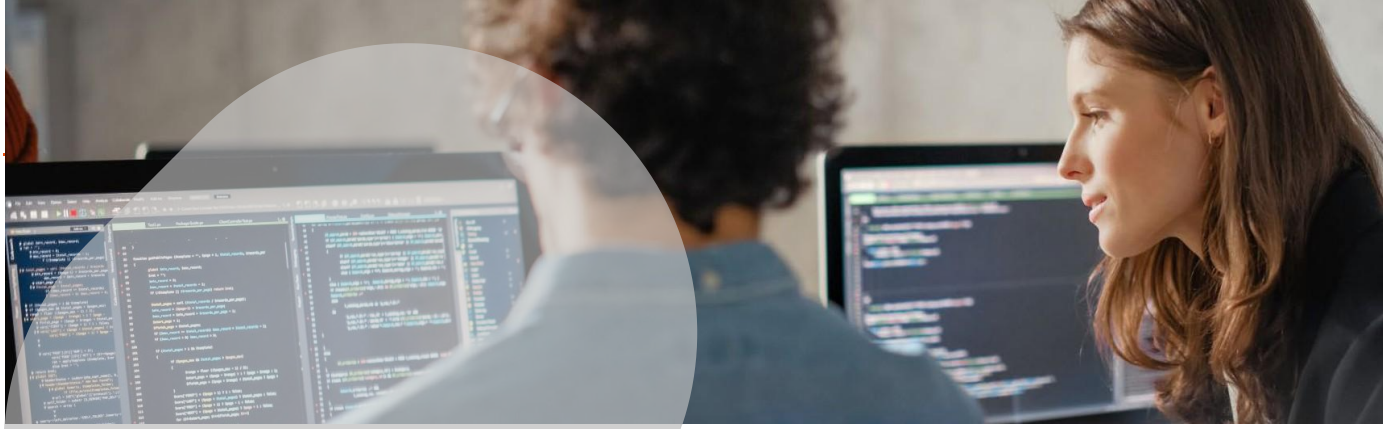
Shoenfield, J. R. *Mathematical Logic*. Secciones sobre primer orden.

Boolos, Burgess & Jeffrey (2007). *Computability and Logic*.

Huth & Ryan (2004). *Logic in Computer Science*.

Ben-Ari, M. (2012). *Mathematical Logic for Computer Science*.

Harrison, J. (2009). *Handbook of Practical Logic and Automated Reasoning*.



Tema 3

Recurrencia e inducción

En este tema vamos a ver:

3.1. Sucesiones y definiciones recursivas.

3.2. Recurrencias lineales.

3.3. Inducción.

Veamos unas ligeras ideas introductorias de lo que se va a estudiar en cada punto.

3.1. Sucesiones y definiciones recursivas

Este tema aborda dos pilares fundamentales de las matemáticas discretas y de la informática teórica:

- **Las recurrencias**, que permiten definir secuencias de forma recursiva, es decir, a partir de valores anteriores. Son esenciales en análisis de algoritmos, estructuras recursivas, árboles, diseño de programas y modelado matemático.
- **La inducción,** una técnica de demostración que permite justificar propiedades que se cumplen para todos los números naturales comprobando un caso base y un paso inductivo. La inducción es clave en demostración de correctitud de programas, propiedades estructurales y resolución de recurrencias.

Ambos conceptos están íntimamente relacionados: muchas recurrencias se demuestran o validan mediante inducción.

Una **sucesión** es una lista ordenada de valores, donde a cada número natural n le asociamos un término.



Ejemplos típicos de sucesiones:

- La sucesión de los números naturales: 0, 1, 2, 3…
- La de números pares: 0, 2, 4, 6…
- La de cuadrados: 0, 1, 4, 9…
- Potencias de 2: 1, 2, 4, 8…

Las sucesiones pueden definirse:

- **Explícitamente**: "el término n-ésimo es n^2"
- **Recursivamente**: "$x_0 = 1$, y cada nuevo término es el doble del anterior"

Las definiciones recursivas son muy naturales en informática porque representan procesos dinámicos paso a paso.

3.2. Recurrencias lineales

Una **recurrencia** expresa cómo obtener un término de la sucesión a partir de otros anteriores. Para definir una sucesión mediante recurrencia se necesita:

1. **Valores iniciales**, que actúan como punto de partida.
2. **Una regla recursiva**, que explica cómo obtener cada nuevo término.



Ejemplos sencillos:

- $x_0 = 0$; cada término aumenta en 1 respecto al anterior.
- $x_0 = 0$; cada término suma n al término anterior (acumula la suma).
- Fibonacci: $x_0 = 0$, $x_1 = 1$; cada término se obtiene sumando los dos anteriores.

Una **recurrencia lineal** es aquella en la que cada término se expresa como una combinación lineal de términos anteriores, sin productos entre ellos ni exponentes.



Ejemplos sencillos:

- "El término actual menos dos veces el anterior más cinco veces el de hace dos posiciones es cero."
- La sucesión de Fibonacci, reescrita en forma lineal, también es un caso:
- "Un término es igual a la suma de los dos anteriores."

En este tipo de recurrencias es posible buscar soluciones generales mediante técnicas algebraicas.

Cuando una recurrencia lineal no tiene términos independientes (solo depende de valores previos), se dice que es **homogénea**.

A cada recurrencia lineal homogénea se le asocia un **polinomio característico**. Las raíces de dicho polinomio permiten describir la forma general de la solución.

El método consiste, en esencia, en:

1. Encontrar las raíces del polinomio.
2. Construir una expresión general a partir de ellas.
3. Ajustar los coeficientes para que los valores iniciales coincidan.

Esta técnica unifica gran parte del análisis de sucesiones lineales y es válida tanto en matemáticas como en análisis de algoritmos.

3.3. Inducción

La **inducción matemática** es un método para demostrar afirmaciones sobre números naturales. Funciona en dos pasos:

1. **Caso base**: comprobar que la propiedad es verdadera para el primer número.
2. **Paso inductivo**: suponer que es verdadera para un número cualquiera y demostrar que entonces también lo es para el siguiente.

Si estos dos pasos funcionan, la propiedad queda demostrada para todos los números naturales.

Un ejemplo clásico es demostrar propiedades de sumas de números naturales, como la fórmula de la suma de números impares o la suma de potencias.

El proceso siempre sigue estos pasos:

- Definir claramente cuál es la propiedad que depende de n.
- Verificar que se cumple para el primer valor.
- Asumir que se cumple para un valor general y demostrar el siguiente.

Se introducen sumatorios porque muchas demostraciones mediante inducción trabajan sobre expresiones del tipo “la suma de los primeros n términos es…”.

El sumatorio simplemente representa una suma compacta de varios valores. Después, la inducción sirve para demostrar fórmulas cerradas como:

- La suma de números impares equivale a un cuadrado perfecto.
- La suma de potencias de 2 da una expresión cerrada simple.

Este tipo de ejemplos ayuda a entrenar el proceso inductivo.

La **inducción fuerte** es una variante donde, para demostrar un caso, se pueden usar todos los casos anteriores (no solo el inmediatamente anterior).

Es útil cuando:

- la relación entre n y términos previos es más compleja,
- hay propiedades que dependen de varios valores anteriores,
- hay que demostrar descomposiciones o factorizaciones.

Ejemplos típicos:

- demostrar una fórmula cerrada para ciertas recurrencias,
- justificar que todo número mayor que 1 tiene un divisor primo,
- demostrar que todo número puede escribirse como una potencia de 2 por un impar.

Este tipo de demostraciones aparece en teoría de números, algoritmos y estructuras recursivas.

Lecturas obligatorias

- Tinoco del Valle, Jesús, Hernández Sastoque, Eric, Escorcia Caballero, Edgardo. (2023). *Introducción a la lógica matemática* (1st ed.). Editorial Unimagdalena. Enlace aquí
  - Capítulo 7
- Villalpando Becerra, J. F., & García Sandoval, A. (2014). *Matemáticas discretas: Aplicaciones y ejercicios*. Larousse - Grupo Editorial Patria. Enlace aquí
  - Capítulos 2 (punto 2.7) y 4

Enlaces de interés

Para practicar recurrencias

Resolución de recurrencias lineales:
https://www.symbolab.com

Material teórico

Recurrence and Induction:
https://www.cs.unm.edu/~saia/classes/561-f18/lec/lec-divideAndConquer.pdf

Bibliografía recomendada

Rosen, K. H. *Matemáticas Discretas y sus Aplicaciones*. Capítulos sobre recurrencia e inducción.

Graham, Knuth & Patashnik. *Concrete Mathematics* (capítulos de sumatorias e inducción).

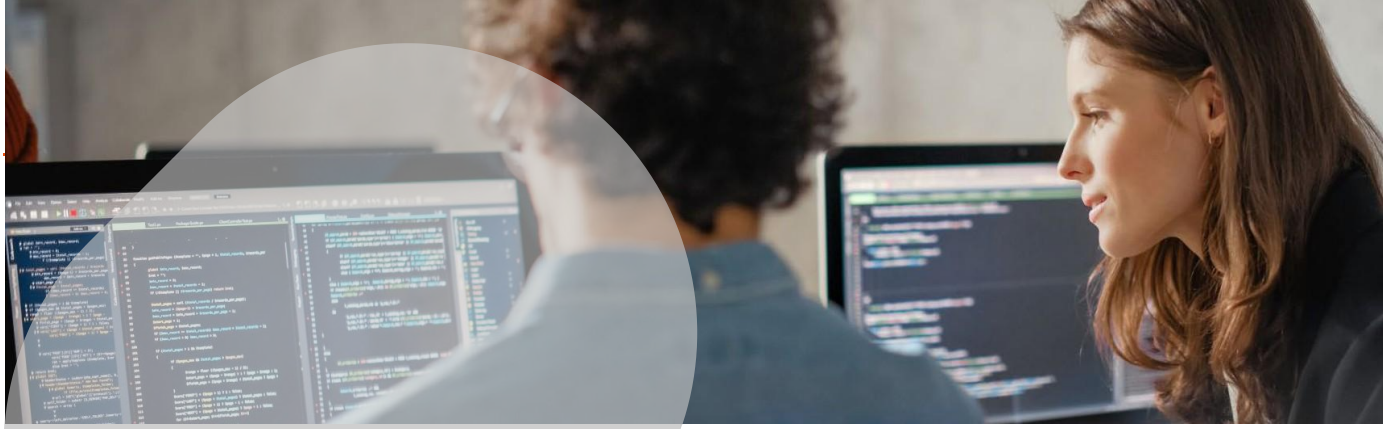
Luenberger. Introduction to Dynamic Systems (para perspectiva de recurrencias lineales simples).

Grimaldi. *Discreet and Combinatorial Mathematics*.

Epp, S. *Discrete Mathematics with Applications*.

Tucker, A. *Applied Combinatorics*.

Kozen, D. *Theory of Computation* (inducción estructural y recursión).



Tema 4

Teoría de grafos

Este tema aborda los siguientes puntos:

4.1. Conceptos básicos de grafos.

4.2. Caminos, ciclos y propiedades de conectividad.

4.3. Árboles, grafos ponderados y árboles generadores.

4.4. Algoritmo de Dijkstra.

Vamos a ver con más detalle qué se va a trabajar en cada subapartado del tema.

4.1. Conceptos básicos de grafos.

Un grafo es una estructura matemática utilizada para representar conexiones entre objetos. Formalmente, un grafo G es una terna:

$$G = (V, E, \gamma)$$

donde:

- V: conjunto de vértices o nodo.
- E: conjunto de aristas o lado.
- $\gamma: E \to E$: aplicación de incidencia, que asigna a cada arista un par de vértices

Ejemplo de arista: $\gamma(e) = \{v_i, v_j\}$ (une el vértice $v_i$con v_j).

Tipos de grafos

a) Grafo simple

Cumple:

1. No hay aristas duplicadas.
2. No tiene bucles: $\gamma(e) = \{v, v'\}$ con $v \neq v'$.

Ejemplo de bucle: una arista que une un vértice consigo mismo.

b) Grafo dirigido (dígrafo)

La aplicación de incidencia es:

$$\gamma: E \to V \times V$$

El par (v_i, v_j) indica dirección (flecha).

Matriz de adyacencia

Para un grafo sin aristas repetidas con vértices $V = \{v_1, \dots, v_n\}$, su **matriz de adyacencia** es:

$$A(i,j) = \begin{cases} 1 & \text{si existe arista entre } v_i \text{ y } v_j, \\ 0 & \text{en otro caso.} \end{cases}$$

Ejemplo de matriz de adyacencia:

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Subgrafos

Un **subgrafo** $H = (V', E', \gamma')$ cumple:

- $V' \subseteq V$
- $E' \subseteq E$
- γ'coincide con γ restringida a E'.

Ejemplo: seleccionar un conjunto de vértices y solo las aristas entre ellos.

4.2. Caminos, ciclos y propiedades de conectividad

Caminos

Un **camino** entre v_1 y v_n es una sucesión de aristas:

$$\{v_1, v_2\}, \{v_2, v_3\}, \ldots, \{v_{n-1}, v_n\}$$

Características:

- **Longitud**: número de aristas.
- **Cerrado** si $v_1 = v_n$.
- **Simple** si no repite vértices.
- **Ciclo**: simple y cerrado.

Ejemplo de ciclo: $\{1,2\}, \{2,4\}, \{4,1\}$

Conectividad

Un grafo es **conexo** si para cualquier par de vértices existe algún camino que los una.

Ejemplo de grafo NO conexo: tiene componentes desconectadas (por ejemplo, un conjunto {1,2,3} y otro {4,5} sin aristas entre ellos).

Caminos y ciclos eulerianos

Son caminos especiales que recorren **todas las aristas exactamente una vez**.

- **Camino euleriano**: abierto.
- **Ciclo euleriano**: cerrado.

Teorema clave

Sea G un grafo:

- Tiene **camino euleriano** ⇔ **exactamente 2 vértices de grado impar**.
- Tiene **ciclo euleriano** ⇔ **todos los vértices tienen grado par**.

(El grado de un vértice es el nº de aristas incidentes).

Ejemplo de grados

Vértice	*Grado*
1	*2*
2	*2*
3	*4*
4	*2*
5	*2*

Como **todos los grados son pares**, entonces **tiene ciclo euleriano**.

4.3. Árboles, grafos ponderados y árboles generadores

Un **árbol** es un grafo:

1. **Conexo**
2. **Sin ciclos**

Propiedad importante:

Entre cualquier par de vértices de un árbol hay un único camino.

Un **árbol generador** es un subgrafo que:

- contiene **todos los vértices**,
- es **un árbol** (conexo y sin ciclos).

Los algoritmos como **Dijkstra**, **Prim** y **Kruskal** se basan en árboles generadores.

Un grafo ponderado incorpora una función:

$$p: E \to [0, +\infty[$$

donde $p(e)$ indica el peso (longitud, coste, distancia…).

4.4. Algoritmo de Dijkstra

El algoritmo de Dijkstra sirve para encontrar el **camino más corto** entre dos vértices en un grafo ponderado sin pesos negativos.

Idea del algoritmo

1. Se asigna:

$$p(v_1) = 0, p(v_j) = \infty \ (j \neq 1)$$

2. En cada iteración se selecciona el vértice **no visitado** con menor peso.
3. Se actualizan los valores:

$$p(v_j) = \min \{p(v_j), p(v_i) + p(v_i, v_j)\}$$

4. Se construye un **árbol de caminos mínimos**.
5. Al finalizar, se reconstruye el camino más corto entre v_1 y v_n.

Lecturas obligatorias

- Villalpando Becerra, J. F., & García Sandoval, A. (2014). *Matemáticas discretas: Aplicaciones y ejercicios*. Larousse - Grupo Editorial Patria. Enlace aquí
  - Capítulos 6 y 7
- Garcia, I. G., & Saldarriaga, Y. M. Z. (2024). *Introducción a la teoría de grafos: Conceptos, algoritmos y aplicaciones* (1st ed.). Editorial Universidad del Norte. Enlace aquí
  - Capítulos 2, 3 y 6

Enlaces de interés

Visualizador interactivo de grafos:
https://csacademy.com/app/graph_editor

Explicación visual de Dijkstra:
https://www.youtube.com/watch?v=GazC3A4OQTE

Curso básico de grafos (MIT OCW):
https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science

Bibliografía recomendada

Rosen, K. H. – *Matemáticas Discretas y sus Aplicaciones*
- Capítulo 10: Teoría de Grafos
- Capítulo 11: Árboles
- Capítulo 13: Algoritmos de caminos mínimos

Diestel, R. – *Graph Theory* (Springer).

West, D. – *Introduction to Graph Theory*.

Bondy & Murty – *Graph Theory with Applications*.

Chartrand & Zhang – *Introduction to Graph Theory*.



Autora
Inmaculada Gema Garcés Andrés

Reservados todos los derechos©
Universidad Internacional de Valencia - 2025

Universidad Internacional de Valencia